CASE STUDY 7 – RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

Design a Python program that generates an itemized retail invoice for several products. The program calculates the amount of each item, applies a threshold-based discount with a maximum concession limit, shares the discount among the items, applies category-specific tax rates and displays the final bill.

2. Objectives

1. Store product details (item, category, price, quantity) in a list of tuples.
2. Calculate the amount of each item and the invoice subtotal.
3. Apply a 10% discount only when the subtotal reaches the discount threshold.
4. Restrict the discount to the maximum concession limit.
5. Apply category-specific tax rates on the discounted amount of every item.
6. Display a neatly formatted itemized invoice and the final bill.

3. Assumptions

- Product data is sample data stored inside the program.
- The discount is 10% of the subtotal when the subtotal is at least ₹1,000, and it never exceeds ₹500.
- The discount is shared among the items in proportion to their amounts, and tax is calculated after the discount.
- Tax rates: Electronics 18%, Clothing 5%, Grocery 2% and 12% for any other category.
- All amounts are in rupees.

4. Algorithm

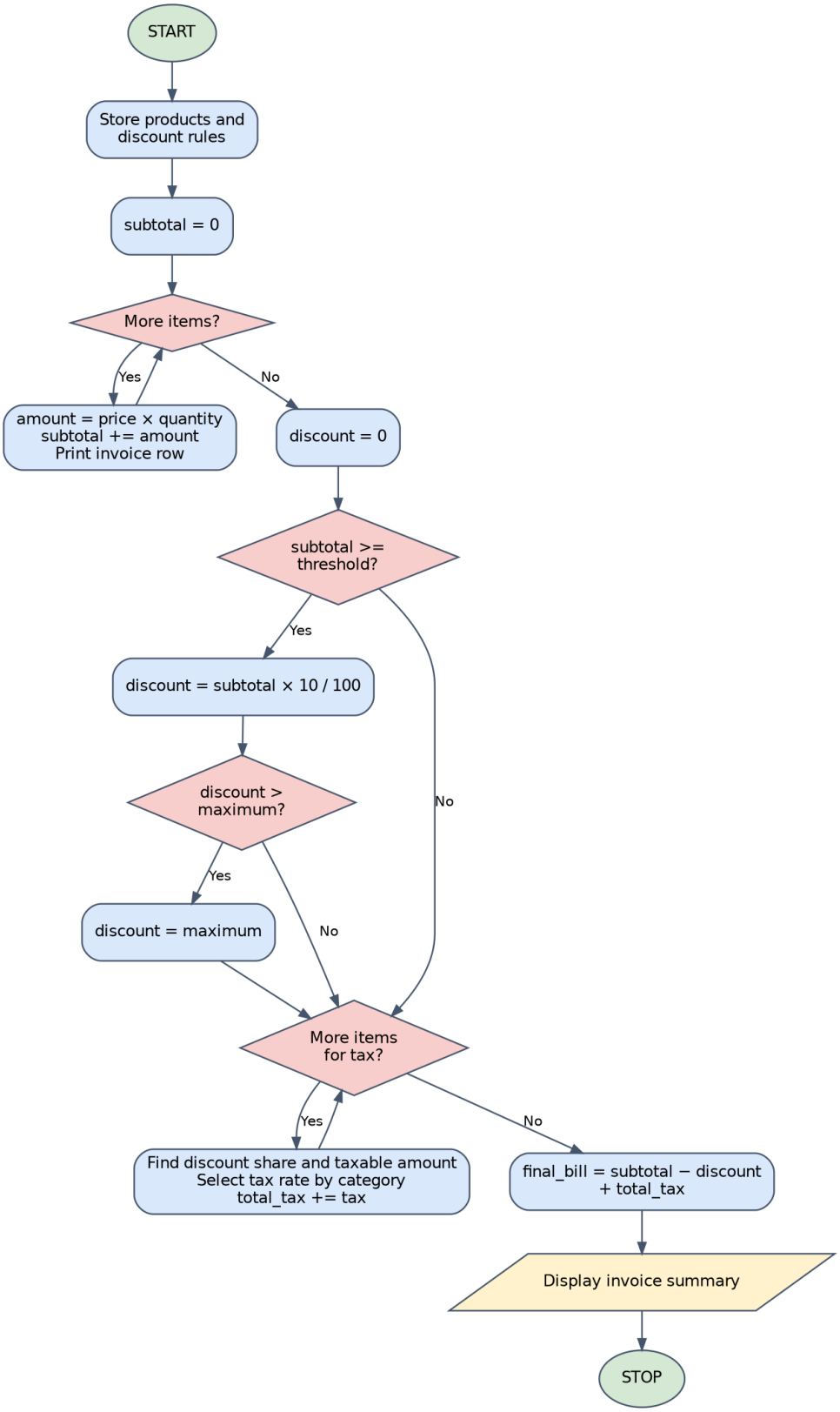
1. Store the products as tuples in a list and set the discount rules.
2. For every product, calculate amount = price × quantity, add it to the subtotal and print the invoice row.
3. If the subtotal is at least 1000, calculate the discount as 10% of the subtotal; if the discount is more than 500, set it to 500.
4. For every product, find its share of the discount and its taxable amount.
5. Select the tax rate with if-elif-else using the category and calculate the tax.
6. Add all item taxes to get the total tax.
7. Calculate final bill = subtotal − discount + total tax.
8. Display the subtotal, discount, total tax and final bill.

5. Pseudocode

```
START
  SET discount_threshold = 1000, discount_percent = 10, max_discount = 500
  STORE products as (item, category, price, quantity)
  subtotal = 0
  FOR each product
    amount = price * quantity
    subtotal = subtotal + amount
    PRINT invoice row
  END FOR
  discount = 0
  IF subtotal >= discount_threshold THEN
    discount = subtotal * discount_percent / 100
    IF discount > max_discount THEN
      discount = max_discount
    END IF
  END IF
  total_tax = 0
  FOR each product
    item_discount = amount / subtotal * discount
    taxable = amount - item_discount
    IF category is Electronics THEN rate = 18
```

```
    ELSE IF category is Clothing THEN rate = 5
    ELSE IF category is Grocery THEN rate = 2
    ELSE rate = 12
    tax = taxable * rate / 100
    total_tax = total_tax + tax
  END FOR
  final_bill = subtotal - discount + total_tax
  PRINT subtotal, discount, total_tax, final_bill
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 7: Retail Billing with Multiple Discount Rules

# (Item, Category, Unit Price, Quantity)
products = [
    ("Laptop", "Electronics", 50000, 1),
    ("Shirt", "Clothing", 1000, 2),
    ("Rice Bag", "Grocery", 500, 2),
    ("Notebook", "Stationery", 50, 4)
]

# Billing rules (project assumptions)
discount_threshold = 1000
discount_percent = 10
max_discount = 500

subtotal = 0
print("RETAIL INVOICE")
print("-" * 51)
print(f"{'Item':<12}{'Category':<14}{'Price':>8}{'Qty':>5}{'Amount':>12}")
print("-" * 51)

for item, category, price, quantity in products:
    amount = price * quantity
    subtotal += amount
    print(f"{item:<12}{category:<14}{price:>8}{quantity:>5}{amount:>12.2f}")

# Threshold discount with a maximum concession limit
discount = 0
if subtotal >= discount_threshold:
    discount = subtotal * discount_percent / 100
    if discount > max_discount:
        discount = max_discount

# Category-wise tax on the discounted amount
total_tax = 0
print()
print("TAX DETAILS")
print("-" * 51)
print(f"{'Item':<12}{'Discount':>10}{'Taxable':>12}{'Rate':>7}{'Tax':>10}")

for item, category, price, quantity in products:
    amount = price * quantity
    item_discount = amount / subtotal * discount
    taxable = amount - item_discount

    if category == "Electronics":
        rate = 18
    elif category == "Clothing":
        rate = 5
    elif category == "Grocery":
        rate = 2
    else:
        rate = 12

    tax = taxable * rate / 100
    total_tax += tax
    print(f"{item:<12}{item_discount:>10.2f}{taxable:>12.2f}{rate:>6}%{tax:>10.2f}")

final_bill = subtotal - discount + total_tax
print("-" * 51)
print(f"{'Subtotal':<39}{subtotal:>12.2f}")
print(f"{'Discount':<39}{discount:>12.2f}")
print(f"{'Total Tax':<39}{total_tax:>12.2f}")
print("=" * 51)
print(f"{'FINAL BILL (Rs.)':<39}{final_bill:>12.2f}")
```

8. Test Cases

Test Case	Input / Scenario	Expected Result
1	Subtotal ₹500 (10 notebooks at ₹50)	Discount ₹0.00 (below threshold)
2	Subtotal exactly ₹1,000	Discount ₹100.00 (10%)
3	Subtotal ₹3,000	Discount ₹300.00 (10%)
4	Sample invoice, subtotal ₹53,200	Discount capped at ₹500.00
5	One Electronics item of ₹100	Tax ₹18.00 (18%)
6	One Clothing item of ₹100	Tax ₹5.00 (5%)
7	One Grocery item of ₹100	Tax ₹2.00 (2%)
8	One item of another category (Stationery) of ₹100	Tax ₹12.00 (12%)

9. Sample Output

```
RETAIL INVOICE
--------------------------------------------------
Item        Category         Price  Qty      Amount
--------------------------------------------------
Laptop      Electronics      50000    1    50000.00
Shirt       Clothing          1000    2     2000.00
Rice Bag    Grocery            500    2     1000.00
Notebook    Stationery          50    4      200.00

TAX DETAILS
--------------------------------------------------
Item          Discount     Taxable   Rate       Tax
Laptop          469.92    49530.08    18%   8915.41
Shirt            18.80     1981.20     5%     99.06
Rice Bag          9.40      990.60     2%     19.81
Notebook          1.88      198.12    12%     23.77
--------------------------------------------------
Subtotal                                   53200.00
Discount                                     500.00
Total Tax                                   9058.06
==================================================
FINAL BILL (Rs.)                           61758.06
```

10. Conclusion

The Retail Billing program prepares a complete itemized invoice using lists of tuples, tuple unpacking, arithmetic and assignment operators, if-elif-else statements, for loops and formatted strings. It shows how threshold discounts, maximum limits and category-wise taxes can be combined to calculate a correct final bill.

CASE STUDY 12 – ATTENDANCE RECOVERY COMPUTATION

1. Problem Statement

Design a Python program that calculates a student's current attendance percentage and checks the 75% eligibility requirement. When attendance is short, the program finds the minimum number of consecutive classes the student must attend to reach the requirement. When attendance is already sufficient, it finds how many future classes can still be missed safely.

2. Objectives

1. Read the number of classes conducted and attended.
2. Validate the input values.
3. Calculate the current attendance percentage.
4. Check eligibility against the required percentage.
5. Use a while loop to find the minimum consecutive classes needed for recovery.
6. Use a second scenario to find how many classes can still be missed.

3. Assumptions

• The required attendance is 75%.
• In the recovery scenario the student attends every future class.
• In the safe-miss scenario the student attends none of the counted future classes.
• Classes conducted and attended are whole numbers.
• An attendance of exactly 75.00% satisfies the requirement.

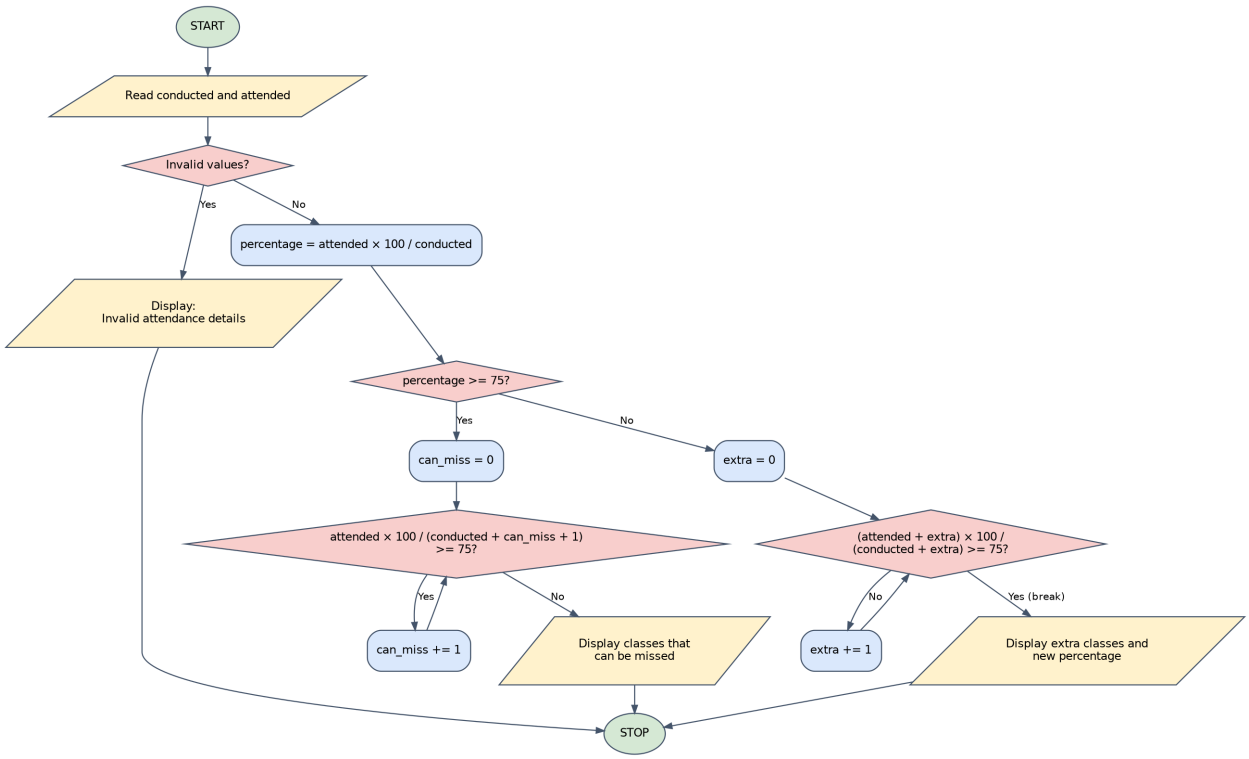
4. Algorithm

1. Read classes conducted and classes attended.
2. If conducted ≤ 0, attended < 0 or attended > conducted, display an error and stop.
3. Calculate percentage = attended × 100 / conducted and display it.
4. If the percentage is at least 75, start can_miss = 0 and increase it while the percentage after one more missed class stays at least 75.
5. Otherwise start extra = 0 and increase it by 1 until (attended + extra) × 100 / (conducted + extra) reaches 75 (use break).
6. Display the number of classes to attend (or to miss) and the new percentage.

5. Pseudocode

```
START
  SET required = 75
  READ conducted, attended
  IF conducted <= 0 OR attended < 0 OR attended > conducted THEN
    PRINT "Invalid attendance details"
  ELSE
    percentage = attended * 100 / conducted
    PRINT percentage
    IF percentage >= required THEN
      can_miss = 0
      WHILE attended * 100 / (conducted + can_miss + 1) >= required
        can_miss = can_miss + 1
      END WHILE
      PRINT can_miss
    ELSE
      extra = 0
      LOOP
        new_percentage = (attended + extra) * 100 / (conducted + extra)
        IF new_percentage >= required THEN BREAK
        extra = extra + 1
      END LOOP
      PRINT extra, new_percentage
    END IF
  END IF
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 12: Attendance Recovery Computation
required = 75

conducted = int(input("Classes conducted so far: "))
attended = int(input("Classes attended so far : "))

if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance details")
else:
    percentage = attended * 100 / conducted
    print(f"Current attendance: {percentage:.2f}%")

    if percentage >= required:
        print("Status: ELIGIBLE (required attendance is already met)")

        # Scenario: how many future classes can be missed safely?
        can_miss = 0
        while attended * 100 / (conducted + can_miss + 1) >= required:
            can_miss += 1
        print("Classes that can still be missed:", can_miss)
    else:
        print("Status: SHORTAGE")

        # Scenario: attend every future class until the target is reached
        extra = 0
        while True:
            new_percentage = (attended + extra) * 100 / (conducted + extra)
            if new_percentage >= required:
                break
            extra += 1

        print("Consecutive classes to attend:", extra)
        print(f"Attendance after recovery: {new_percentage:.2f}%")
```

8. Test Cases

Test Case	Classes Conducted / Attended	Expected Result
1	20 / 16	80.00%, eligible; 1 more class can be missed
2	20 / 15	75.00%, eligible; 0 classes can be missed
3	20 / 10	50.00%, shortage; attend 20 consecutive classes (new 75.00%)
4	40 / 28	70.00%, shortage; attend 8 consecutive classes (new 75.00%)
5	0 / 0	Invalid attendance details
6	10 / 12	Invalid attendance details (attended > conducted)

9. Sample Output

Sample Run 1 (input: 20, 10)

```
Classes conducted so far: 20
Classes attended so far : 10
Current attendance: 50.00%
Status: SHORTAGE
Consecutive classes to attend: 20
Attendance after recovery: 75.00%
```

Sample Run 2 (input: 20, 16)

```
Classes conducted so far: 20
Classes attended so far : 16
Current attendance: 80.00%
Status: ELIGIBLE (required attendance is already met)
Classes that can still be missed: 1
```

Sample Run 3 (input: 10, 12)

```
Classes conducted so far: 10
Classes attended so far : 12
Invalid attendance details
```

10. Conclusion

The Attendance Recovery program calculates attendance using arithmetic and relational operators, validates input with logical operators and uses while loops with break to search for the smallest number of classes that satisfies the eligibility rule. It also shows how loops can answer what-if questions about future attendance.

CASE STUDY 13 – ATM TRANSACTION SEQUENCE SIMULATION

1. Problem Statement

Design a Python program that simulates a sequence of ATM transactions. The program maintains the account balance, enforces a per-transaction withdrawal limit and a daily withdrawal limit, rejects invalid transactions and prints a summary of the accepted and rejected transactions.

2. Objectives

1. Store transactions as (type, amount) tuples in a list.
2. Maintain a running account balance.
3. Accept valid deposits and withdrawals.
4. Reject invalid amounts, unknown transaction types and withdrawals that break a limit or the balance.
5. Count accepted and rejected transactions.
6. Display a transaction log and a final summary.

3. Assumptions

• The opening balance is ₹5,000.
• A single withdrawal cannot exceed ₹2,000 and the total withdrawn in one day cannot exceed ₹5,000.
• Only 'deposit' and 'withdraw' are valid transaction types.
• The amount of a transaction must be greater than zero.
• The balance changes only for accepted transactions.

4. Algorithm

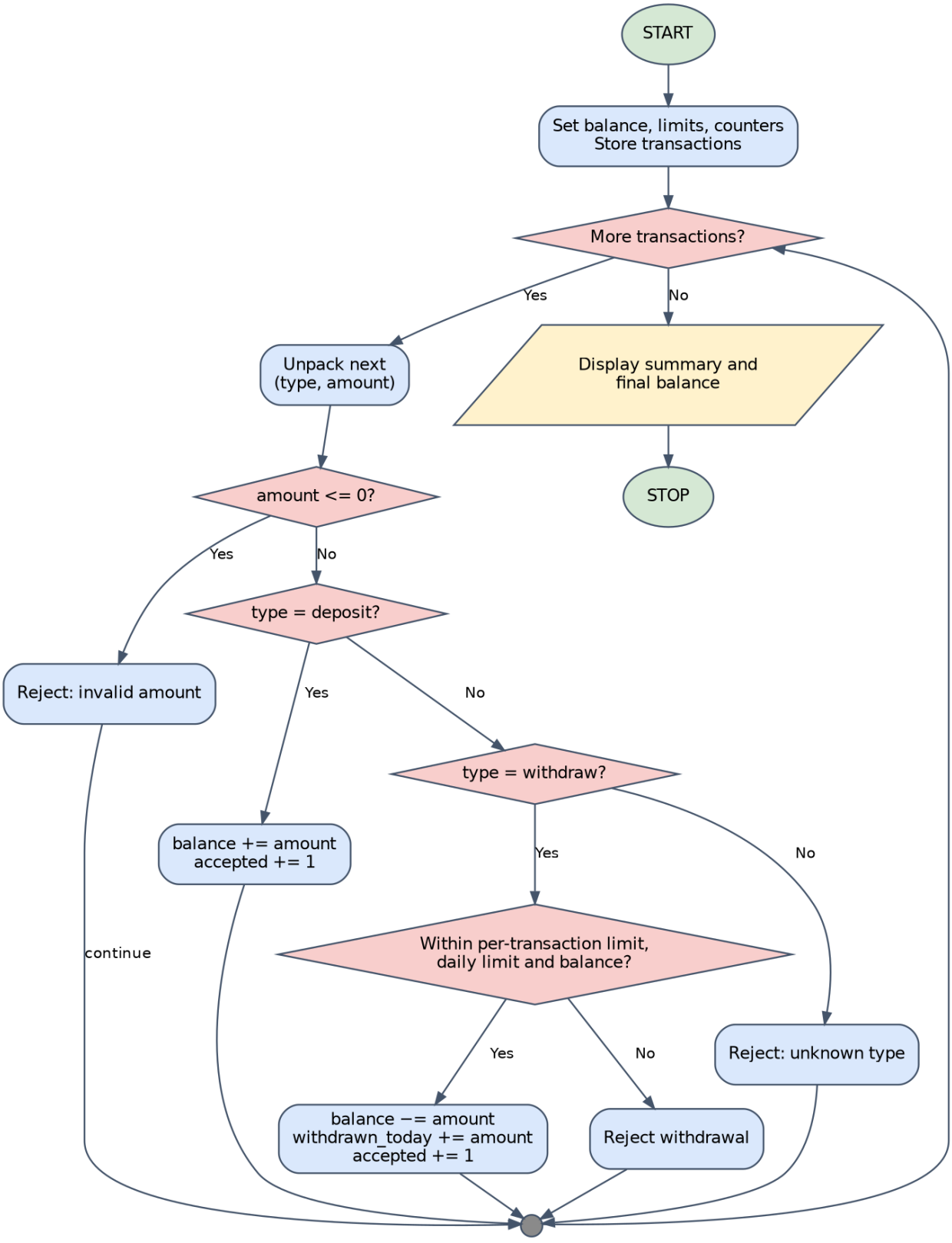
1. Set the opening balance, limits and counters; store the transactions as tuples.
2. Take the next transaction and unpack it into type and amount.
3. If the amount is zero or negative, reject it and continue with the next transaction.
4. If the type is deposit, add the amount to the balance and count it as accepted.
5. If the type is withdraw, reject it when it exceeds the per-transaction limit, the daily limit or the balance; otherwise subtract it.
6. Reject any other transaction type.
7. Repeat for all transactions and display the summary.

5. Pseudocode

```
START
  balance = 5000
  per_transaction_limit = 2000
  daily_limit = 5000
  withdrawn_today = 0
  accepted = 0, rejected = 0
  STORE transactions as (type, amount)
  FOR each (type, amount) in transactions
    IF amount <= 0 THEN
      PRINT rejected (invalid amount)
      rejected = rejected + 1
      CONTINUE
    END IF
    IF type is "deposit" THEN
      balance = balance + amount
      accepted = accepted + 1
    ELSE IF type is "withdraw" THEN
      IF amount > per_transaction_limit THEN reject
      ELSE IF withdrawn_today + amount > daily_limit THEN reject
      ELSE IF amount > balance THEN reject
      ELSE
        balance = balance - amount
        withdrawn_today = withdrawn_today + amount
        accepted = accepted + 1
      END IF
```

```
    ELSE
      reject (unknown type)
    END IF
  END FOR
  PRINT accepted, rejected, total deposited, total withdrawn, balance
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 13: ATM Transaction Sequence Simulation
balance = 5000
per_transaction_limit = 2000
daily_limit = 5000

withdrawn_today = 0
total_deposited = 0
total_withdrawn = 0
accepted = 0
rejected = 0
number = 0

# (Transaction Type, Amount)
transactions = [
    ("deposit", 1000),
    ("withdraw", 1500),
    ("withdraw", 2500),
    ("deposit", 0),
    ("withdraw", 2000),
    ("withdraw", 1800),
    ("transfer", 500),
    ("deposit", 700)
]

print("ATM TRANSACTION LOG")
print("Opening balance:", balance)
print("-" * 62)

for transaction_type, amount in transactions:
    number += 1

    if amount <= 0:
        print(f"{number}. {transaction_type} {amount} -> REJECTED (invalid amount)")
        rejected += 1
        continue

    if transaction_type == "deposit":
        balance += amount
        total_deposited += amount
        accepted += 1
        print(f"{number}. deposit {amount} -> ACCEPTED | Balance: {balance}")
    elif transaction_type == "withdraw":
        if amount > per_transaction_limit:
            print(f"{number}. withdraw {amount} -> REJECTED (per-transaction limit exceeded)")
            rejected += 1
        elif withdrawn_today + amount > daily_limit:
            print(f"{number}. withdraw {amount} -> REJECTED (daily limit exceeded)")
            rejected += 1
        elif amount > balance:
            print(f"{number}. withdraw {amount} -> REJECTED (insufficient balance)")
            rejected += 1
        else:
            balance -= amount
            withdrawn_today += amount
            total_withdrawn += amount
            accepted += 1
            print(f"{number}. withdraw {amount} -> ACCEPTED | Balance: {balance}")
    else:
        print(f"{number}. {transaction_type} {amount} -> REJECTED (unknown transaction type)")
        rejected += 1

print("-" * 62)
print("TRANSACTION SUMMARY")
print("Accepted transactions :", accepted)
print("Rejected transactions :", rejected)
print("Total deposited       :", total_deposited)
print("Total withdrawn       :", total_withdrawn)
print("Final balance         :", balance)
```

8. Test Cases

Test Case	Transaction	Expected Result
1	deposit ₹1,000	Accepted; balance ₹6,000
2	withdraw ₹1,500	Accepted; balance ₹4,500
3	withdraw ₹2,500	Rejected: per-transaction limit exceeded
4	deposit ₹0	Rejected: invalid amount
5	withdraw ₹2,000	Accepted; balance ₹2,500 (withdrawn today ₹3,500)
6	withdraw ₹1,800	Rejected: daily limit exceeded (3,500 + 1,800 > 5,000)
7	transfer ₹500	Rejected: unknown transaction type
8	deposit ₹700	Accepted; final balance ₹3,200

9. Sample Output

```
ATM TRANSACTION LOG
Opening balance: 5000
------------------------------------------------------------
1. deposit 1000 -> ACCEPTED | Balance: 6000
2. withdraw 1500 -> ACCEPTED | Balance: 4500
3. withdraw 2500 -> REJECTED (per-transaction limit exceeded)
4. deposit 0 -> REJECTED (invalid amount)
5. withdraw 2000 -> ACCEPTED | Balance: 2500
6. withdraw 1800 -> REJECTED (daily limit exceeded)
7. transfer 500 -> REJECTED (unknown transaction type)
8. deposit 700 -> ACCEPTED | Balance: 3200
------------------------------------------------------------
TRANSACTION SUMMARY
Accepted transactions : 4
Rejected transactions : 4
Total deposited       : 1700
Total withdrawn       : 3500
Final balance         : 3200
```

10. Conclusion

The ATM simulation uses tuples, tuple unpacking, for loops, continue and nested conditional statements to validate every transaction before changing the balance. It demonstrates how relational and logical operators can enforce banking rules such as amount limits, daily limits and insufficient balance.

CASE STUDY 17 – STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Design a Python program that compares the scores of students across three examinations in several subjects. The program calculates the improvement or decline between examinations, identifies consistently weak subjects and classifies every student according to the final examination average.

2. Objectives

1. Store examination scores of every subject in nested lists.
2. Calculate the change between consecutive examinations.
3. Find the overall trend of every subject (improved, declined or no change).
4. Identify subjects in which a student is consistently weak.
5. Classify each student using the final examination average.
6. Display a formatted report for every student.

3. Assumptions

- Scores are out of 100 and three examinations (E1, E2, E3) are conducted.
- Overall trend is decided by comparing E3 with E1.
- A subject is consistently weak when the score is below 50 in all three examinations.
- Classification uses the average of E3 scores: 75 or more is Good, 50 to 74.99 is Average and below 50 is Needs Improvement.
- Sample student data is stored in the program.

4. Algorithm

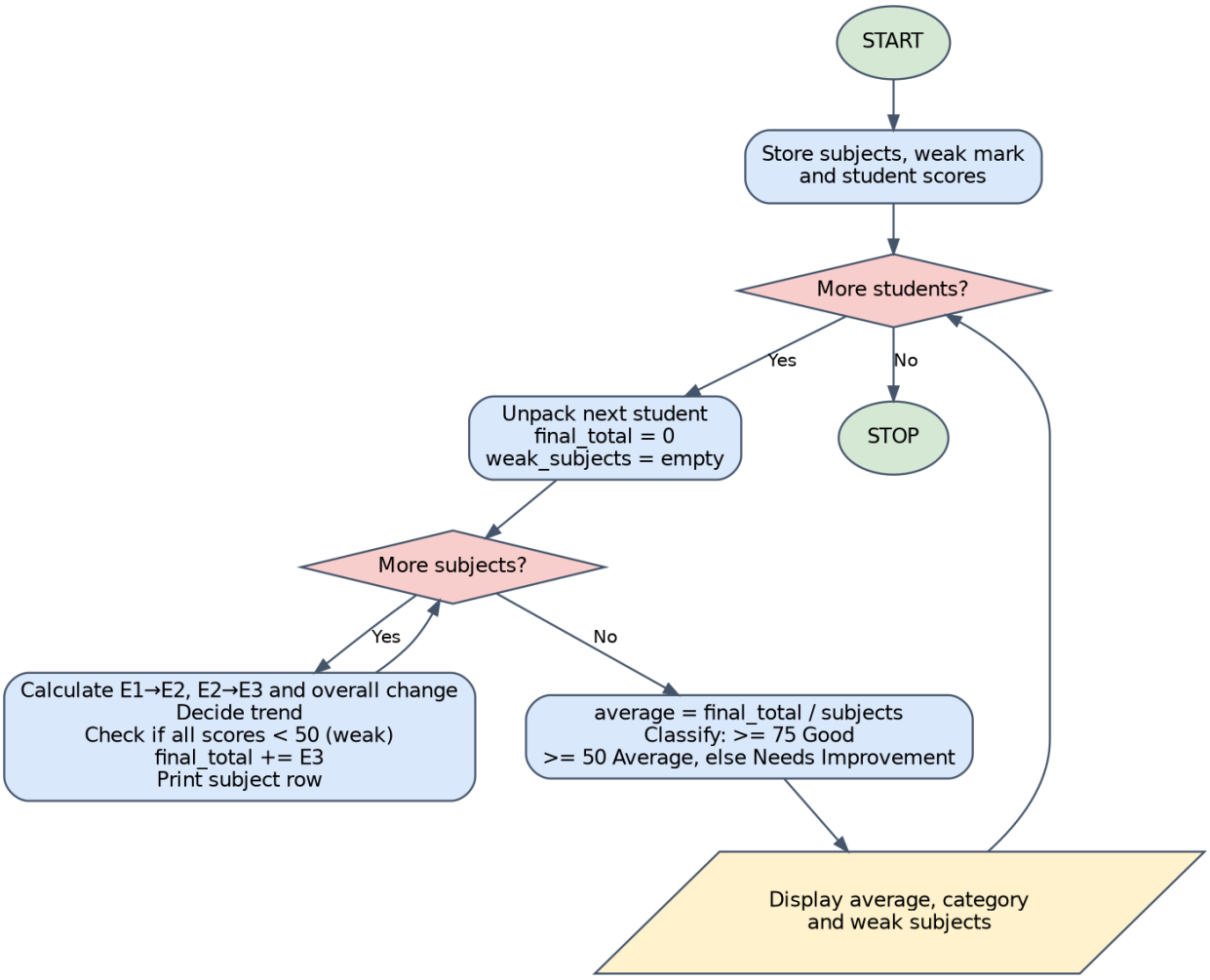
1. Store the subject names and, for every student, a nested list of three scores per subject.
2. Take the next student and unpack the name and scores.
3. For every subject calculate the E1→E2 change, the E2→E3 change and the overall change.
4. Decide the trend using if-elif-else.
5. Check with a loop whether all three scores are below 50 and, if so, add the subject to the weak list.
6. Add the E3 score to the final total.
7. Calculate the average, classify the student and display the report.
8. Repeat for all students.

5. Pseudocode

```
START
  STORE subjects and weak_mark = 50
  STORE students as (name, scores) with scores = [[E1, E2, E3], ...]
  FOR each (name, all_scores) in students
    final_total = 0
    weak_subjects = empty list
    FOR each subject index i
      change1 = E2 - E1
      change2 = E3 - E2
      overall = E3 - E1
      IF overall > 0 THEN trend = "Improved"
      ELSE IF overall < 0 THEN trend = "Declined"
      ELSE trend = "No change"
      weak = TRUE
      FOR each score of the subject
        IF score >= weak_mark THEN weak = FALSE
      END FOR
      IF weak THEN add subject to weak_subjects
      final_total = final_total + E3
      PRINT subject row
    END FOR
    average = final_total / number of subjects
    IF average >= 75 THEN category = "Good"
    ELSE IF average >= 50 THEN category = "Average"
    ELSE category = "Needs Improvement"
    PRINT average, category, weak_subjects
```

```
  END FOR
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 17: Student Performance Trend Analysis
subjects = ["Python", "Mathematics", "Web Development"]
weak_mark = 50

# (Student, [[Exam1, Exam2, Exam3] for every subject])
students = [
    ("Anu", [[70, 75, 82], [60, 58, 65], [55, 62, 68]]),
    ("Bala", [[85, 88, 90], [78, 80, 76], [90, 92, 95]]),
    ("Chandu", [[45, 40, 48], [38, 42, 35], [60, 55, 58]])
]

for name, all_scores in students:
    print("STUDENT:", name)
    print("-" * 66)
    print(f"{'Subject':<17}{'E1':>4}{'E2':>4}{'E3':>4}{'E1-E2':>8}{'E2-E3':>8}   Trend")

    final_total = 0
    weak_subjects = []

    for i in range(len(subjects)):
        scores = all_scores[i]
        change1 = scores[1] - scores[0]
        change2 = scores[2] - scores[1]
        overall = scores[2] - scores[0]

        if overall > 0:
            trend = "Improved"
        elif overall < 0:
            trend = "Declined"
        else:
```

```
            trend = "No change"

        # A subject is consistently weak if every score is below the weak mark
        weak = True
        for score in scores:
            if score >= weak_mark:
                weak = False
        if weak:
            weak_subjects.append(subjects[i])

        final_total += scores[2]
        print(f"{subjects[i]:<17}{scores[0]:>4}{scores[1]:>4}{scores[2]:>4}{change1:>+8}{change2:>+8}   {trend}")

    average = final_total / len(subjects)

    if average >= 75:
        category = "Good"
    elif average >= 50:
        category = "Average"
    else:
        category = "Needs Improvement"

    print(f"Final exam average : {average:.2f}")
    print("Performance        :", category)
    if len(weak_subjects) > 0:
        print("Consistently weak  :", ", ".join(weak_subjects))
    else:
        print("Consistently weak  : None")
    print()
```

8. Test Cases

Test Case	Student / Data	Expected Result
1	Anu - Python 70, 75, 82	Improved (+5, +7)
2	Anu - final average (82, 65, 68)	71.67 - Average; no consistently weak subject
3	Bala - Mathematics 78, 80, 76	Declined overall (−2): +2 then −4
4	Bala - final average (90, 76, 95)	87.00 - Good
5	Chandu - Python 45, 40, 48 and Mathematics 38, 42, 35	Below 50 in every examination - consistently weak
6	Chandu - final average (48, 35, 58)	47.00 - Needs Improvement

9. Sample Output

```
STUDENT: Anu
------------------------------------------------------------------
Subject            E1  E2  E3   E1-E2   E2-E3   Trend
Python             70  75  82      +5      +7   Improved
Mathematics        60  58  65      -2      +7   Improved
Web Development    55  62  68      +7      +6   Improved
Final exam average : 71.67
Performance        : Average
Consistently weak  : None

STUDENT: Bala
------------------------------------------------------------------
Subject            E1  E2  E3   E1-E2   E2-E3   Trend
Python             85  88  90      +3      +2   Improved
Mathematics        78  80  76      +2      -4   Declined
Web Development    90  92  95      +2      +3   Improved
Final exam average : 87.00
Performance        : Good
Consistently weak  : None

STUDENT: Chandu
------------------------------------------------------------------
Subject            E1  E2  E3   E1-E2   E2-E3   Trend
Python             45  40  48      -5      +8   Improved
Mathematics        38  42  35      +4      -7   Declined
Web Development    60  55  58      -5      +3   Declined
Final exam average : 47.00
```

```
Performance        : Needs Improvement
Consistently weak  : Python, Mathematics
```

10. Conclusion

The Student Performance Trend program uses nested lists, list indexing and nested loops to compare scores across examinations. Conditional statements classify trends and performance, and string methods such as join help present the weak subjects clearly in the final report.

CASE STUDY 20 – UNIVERSITY ADMISSION DECISION ENGINE

1. Problem Statement

Design a Python program to evaluate university admission applicants based on subject-wise eligibility, aggregate marks, course-specific minimum thresholds and predefined tie-breaking rules. The program identifies the eligible applicants, ranks them according to their performance and lists the ineligible applicants with the reason.

2. Objectives

1. Check minimum marks in all subjects.
2. Calculate the aggregate percentage.
3. Apply course-specific minimum aggregate requirements.
4. Separate eligible and ineligible applicants.
5. Rank eligible applicants using predefined tie-breaking rules.
6. Display the ranking and the reason for ineligibility.

3. Assumptions

- Three subjects are considered: Mathematics, Physics and Computer Science. The minimum mark in each subject is 50.
- Course thresholds are CSE = 70%, ECE = 65% and EEE = 60%.
- Ties in aggregate are resolved by higher Mathematics marks, then higher Computer Science marks, then alphabetical order of the name.
- Applicant details are sample data stored as tuples.

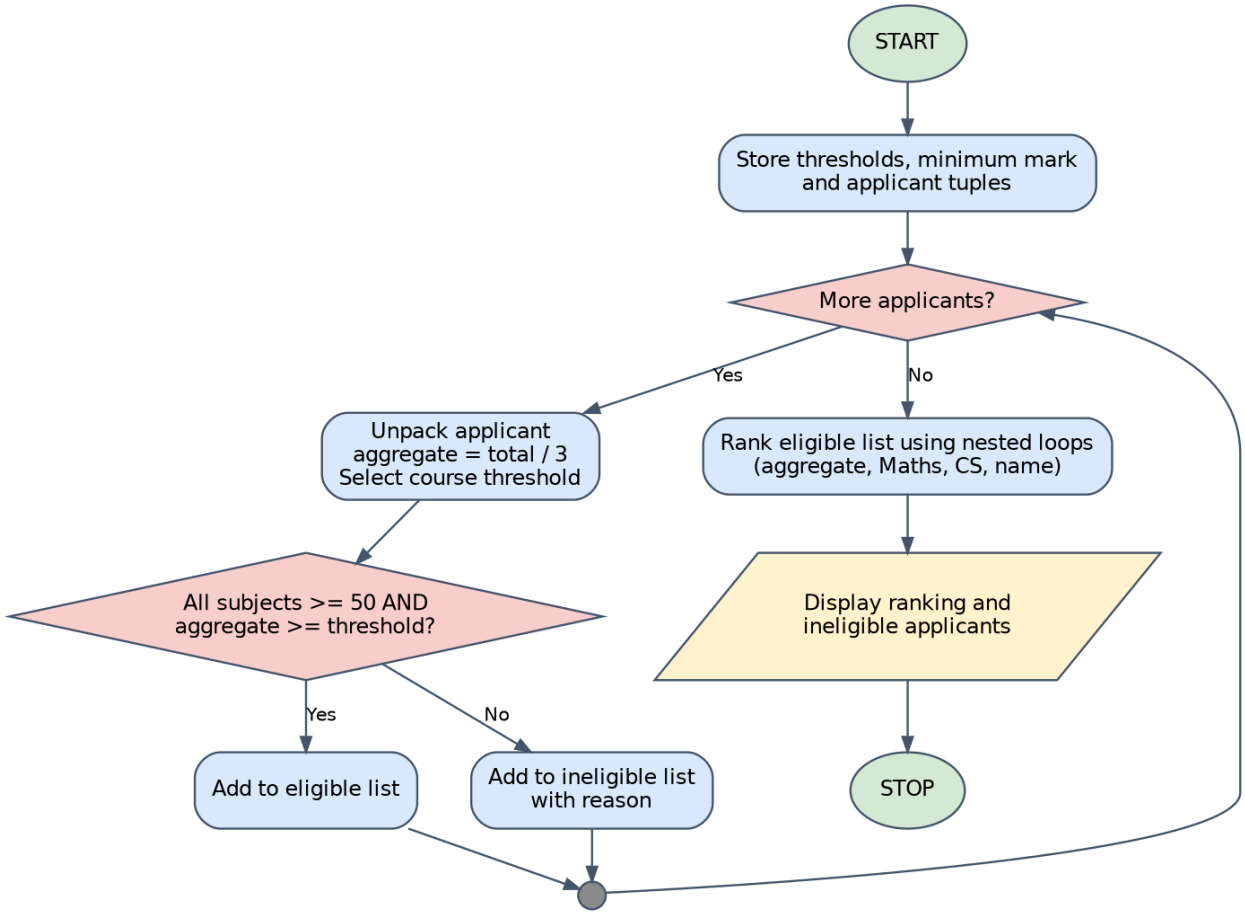
4. Algorithm

1. Store the applicant details in a list of tuples and set the course thresholds.
2. Unpack each applicant and calculate the total and the aggregate.
3. Select the course threshold using if-elif-else.
4. Check the subject-wise minimum marks and the course threshold.
5. Store the applicant in the eligible list, or in the ineligible list with the reason.
6. Rank the eligible applicants using nested loops and the tie-breaking rules.
7. Display the final ranking and the ineligible applicants.

5. Pseudocode

```
START
  SET cse = 70, ece = 65, eee = 60, minimum_subject_mark = 50
  STORE applicants as (name, course, maths, physics, computer)
  eligible = empty list, ineligible = empty list
  FOR each applicant
    UNPACK name, course, maths, physics, computer
    aggregate = (maths + physics + computer) / 3
    SELECT course_threshold using the course
    subjects_ok = all three marks >= minimum_subject_mark
    IF subjects_ok AND aggregate >= course_threshold THEN
      add applicant to eligible
    ELSE
      reason = subject mark below 50 OR aggregate below threshold
      add applicant to ineligible
    END IF
  END FOR
  FOR i = 0 to last eligible position
    FOR j = i + 1 to last eligible position
      IF second is better than first (aggregate, then maths,
         then computer science, then alphabetical name) THEN
        swap the two applicants
      END IF
    END FOR
  END FOR
  PRINT ranking and ineligible applicants
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 20: University Admission Decision Engine
cse = 70
ece = 65
eee = 60
minimum_subject_mark = 50

# (Name, Course, Mathematics, Physics, Computer Science)
applicants = [
    ("Anil", "CSE", 82, 76, 88),
    ("Bhavya", "ECE", 70, 68, 72),
    ("Charan", "CSE", 74, 70, 76),
    ("Divya", "EEE", 65, 60, 70),
    ("Esha", "CSE", 69, 72, 75),
    ("Farhan", "ECE", 70, 68, 72),
    ("Gopal", "CSE", 48, 80, 90),
    ("Hari", "ECE", 55, 60, 62)
]

eligible = []
ineligible = []

for applicant in applicants:
    name, course, maths, physics, computer = applicant
    total = maths + physics + computer
    aggregate = total / 3

    if course == "CSE":
        course_threshold = cse
    elif course == "ECE":
        course_threshold = ece
    else:
        course_threshold = eee

    subjects_ok = (maths >= minimum_subject_mark and
                   physics >= minimum_subject_mark and
                   computer >= minimum_subject_mark)
```

```
    if subjects_ok and aggregate >= course_threshold:
        eligible.append((name, course, maths, physics, computer, aggregate))
    else:
        if not subjects_ok:
            reason = "Subject mark below 50"
        else:
            reason = "Aggregate below course threshold"
        ineligible.append((name, course, aggregate, reason))

# Ranking: aggregate, then Mathematics, then Computer Science, then name
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        first = eligible[i]
        second = eligible[j]
        swap = False

        if first[5] < second[5]:
            swap = True
        elif first[5] == second[5] and first[2] < second[2]:
            swap = True
        elif (first[5] == second[5] and
              first[2] == second[2] and
              first[4] < second[4]):
            swap = True
        elif (first[5] == second[5] and
              first[2] == second[2] and
              first[4] == second[4] and
              first[0] > second[0]):
            swap = True

        if swap:
            eligible[i], eligible[j] = eligible[j], eligible[i]

print("UNIVERSITY ADMISSION RANKING")
print("-" * 62)
print(f"{'Rank':<6}{'Name':<10}{'Course':<8}{'Maths':>6}{'Phy':>6}{'CS':>6}{'Aggregate':>12}")
print("-" * 62)

rank = 1
for name, course, maths, physics, computer, aggregate in eligible:
    print(f"{rank:<6}{name:<10}{course:<8}{maths:>6}{physics:>6}{computer:>6}{aggregate:>11.2f}%")
    rank += 1

print()
print("INELIGIBLE APPLICANTS")
print("-" * 62)
for name, course, aggregate, reason in ineligible:
    print(f"{name:<10}{course:<8}{aggregate:>7.2f}%   {reason}")
```

8. Test Cases

Test Case	Applicant	Course	Expected Result
1	Anil	CSE	Eligible - Rank 1 (aggregate 82.00%)
2	Charan	CSE	Eligible - Rank 2 (aggregate 73.33%)
3	Esha	CSE	Eligible - Rank 3 (aggregate 72.00%, above the CSE threshold of 70%)
4	Bhavya	ECE	Eligible - Rank 4 (70.00%; wins the tie with Farhan alphabetically)
5	Farhan	ECE	Eligible - Rank 5 (same marks as Bhavya)
6	Divya	EEE	Eligible - Rank 6 (aggregate 65.00%)
7	Gopal	CSE	Ineligible - Mathematics 48 is below 50 (aggregate 72.67%)
8	Hari	ECE	Ineligible - aggregate 59.00% is below the ECE threshold of 65%

9. Sample Output

```
UNIVERSITY ADMISSION RANKING
-------------------------------------------------------------
Rank  Name      Course   Maths   Phy    CS   Aggregate
-------------------------------------------------------------
1     Anil      CSE         82    76    88      82.00%
2     Charan    CSE         74    70    76      73.33%
3     Esha      CSE         69    72    75      72.00%
4     Bhavya    ECE         70    68    72      70.00%
5     Farhan    ECE         70    68    72      70.00%
6     Divya     EEE         65    60    70      65.00%

INELIGIBLE APPLICANTS
-------------------------------------------------------------
Gopal     CSE       72.67%   Subject mark below 50
Hari      ECE       59.00%   Aggregate below course threshold
```

10. Conclusion

The University Admission Decision Engine evaluates applicants using subject-wise minimum marks and course-specific aggregate requirements. It separates eligible and ineligible applicants and ranks the eligible candidates using predefined tie-breaking rules. The program demonstrates variables, data types, operators, conditional statements, loops, lists, tuples and tuple unpacking.

CASE STUDY 27 – ADVANCED NUMERIC PATTERN GENERATOR

1. Problem Statement

Design a Python program that produces a hollow numeric diamond pattern for a size entered by the user. Every border position of row r is printed with the number r, the inside of the diamond is left empty, the pattern is aligned carefully and invalid sizes are rejected.

2. Objectives

1. Read the pattern size from the user.
2. Validate that the size is between 1 and 9.
3. Build the sequence of row numbers for the upper and lower halves.
4. Use string repetition to align the leading spaces and the inner gap.
5. Print the hollow numeric diamond.

3. Assumptions

- The size is a whole number from 1 to 9 so that every row number is a single digit and the alignment is not disturbed.
- The first and last rows contain a single number; every other row contains the row number at the left and right borders.
- Rows are numbered 1, 2, ..., size, ..., 2, 1.

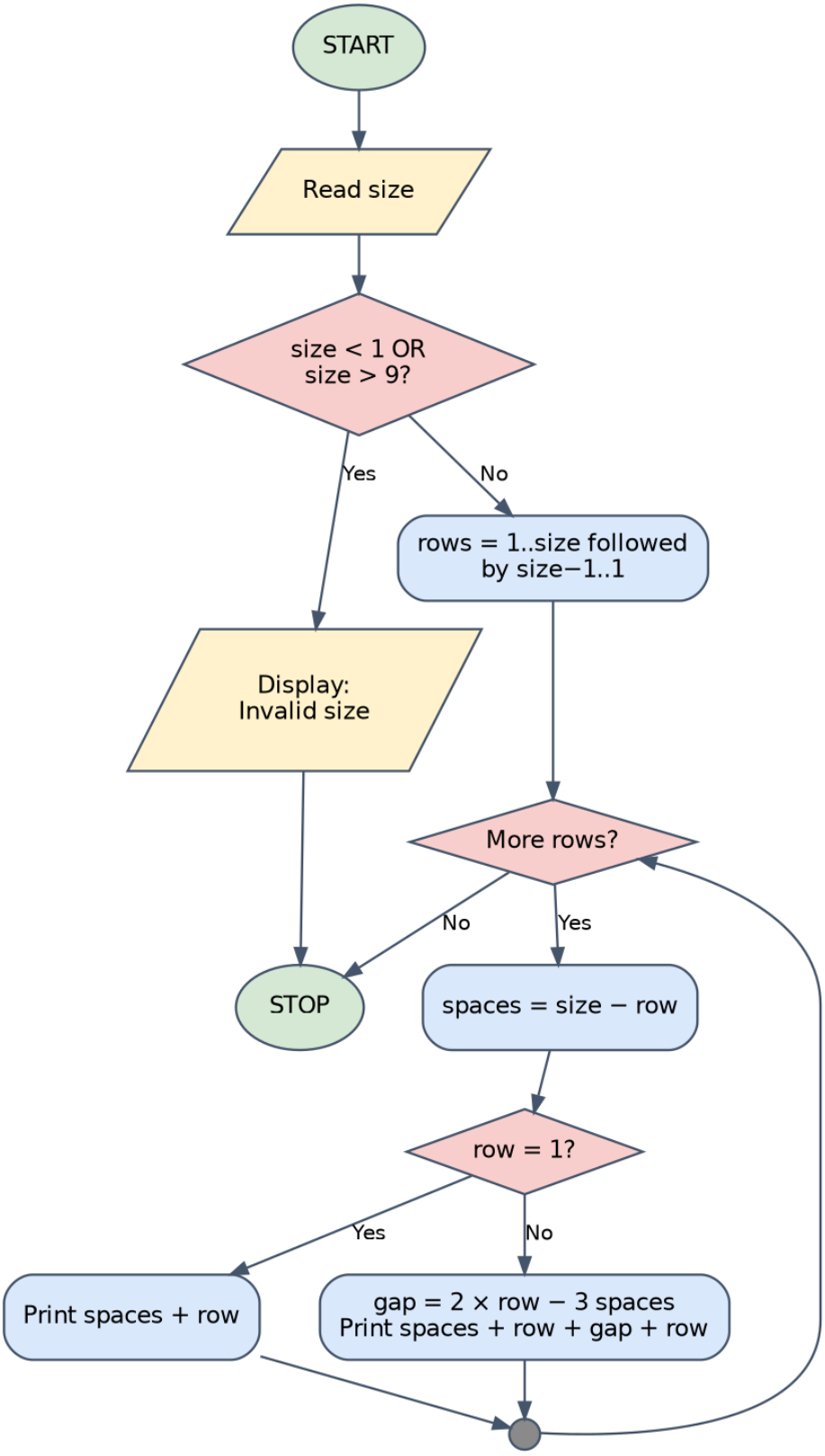
4. Algorithm

1. Read the size n.
2. If n < 1 or n > 9, display an error message and stop.
3. Create the list of row numbers: 1 to n followed by n−1 down to 1.
4. For every row number, print (n − row) leading spaces.
5. If the row number is 1, print only the number.
6. Otherwise print the number, (2 × row − 3) spaces and the number again.

5. Pseudocode

```
START
  READ size
  IF size < 1 OR size > 9 THEN
    PRINT "Invalid size"
  ELSE
    rows = [1, 2, ..., size] followed by [size-1, ..., 2, 1]
    FOR each row in rows
      spaces = (size - row) spaces
      IF row = 1 THEN
        PRINT spaces + row
      ELSE
        gap = (2 * row - 3) spaces
        PRINT spaces + row + gap + row
      END IF
    END FOR
  END IF
STOP
```

6. Flowchart


START
Read size
size < 1 OR
size > 9?
Yes
No
rows = 1..size followed
by size−1..1
Display:
Invalid size
More rows?
No
Yes
STOP
spaces = size − row
row = 1?
Yes
No
Print spaces + row
gap = 2 × row − 3 spaces
Print spaces + row + gap + row

7. Python Source Code

```
# Case Study 27: Advanced Numeric Pattern Generator (Hollow Numeric Diamond)
size = int(input("Enter pattern size (1-9): "))

if size < 1 or size > 9:
    print("Invalid size. Please enter a number from 1 to 9.")
else:
    # Row numbers: 1, 2, ..., size, ..., 2, 1
    rows = list(range(1, size + 1)) + list(range(size - 1, 0, -1))

    for row in rows:
        spaces = " " * (size - row)
        if row == 1:
            print(spaces + str(row))
        else:
            gap = " " * (2 * row - 3)
            print(spaces + str(row) + gap + str(row))
```

8. Test Cases

Test Case	Size (n)	Expected Result
1	1	A single line containing 1
2	2	3 lines: 1, 2 2, 1
3	4	7 lines - hollow diamond with borders 1, 2, 3, 4, 3, 2, 1
4	9	17 lines, perfectly aligned single-digit borders
5	0	Invalid size message
6	10	Invalid size message (above 9)
7	-3	Invalid size message

9. Sample Output

Sample Run 1 (input: 4)

```
Enter pattern size (1-9): 4
   1
  2 2
 3   3
4     4
 3   3
  2 2
   1
```

Sample Run 2 (input: 1)

```
Enter pattern size (1-9): 1
1
```

Sample Run 3 (input: 0)

```
Enter pattern size (1-9): 0
Invalid size. Please enter a number from 1 to 9.
```

10. Conclusion

The Numeric Pattern Generator uses range(), list concatenation, string repetition and conditional statements to print a symmetric hollow diamond. The validation step protects the alignment of the output and shows how pattern generation depends on spaces, gaps and row numbers.

CASE STUDY 29 – PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Design a Python program that evaluates a password against the minimum length, character-category, repetition and prohibited-word rules. The program reports PASS or FAIL for every rule and explains each failed condition.

2. Objectives

1. Check the minimum password length.
2. Check for uppercase, lowercase, digit and special characters.
3. Detect adjacent repeated characters.
4. Reject prohibited words regardless of letter case.
5. Explain every failed rule and display the final result.

3. Assumptions

- The minimum length is 8 characters.
- At least one uppercase letter, one lowercase letter, one digit and one special character are required; any character that is not a letter or digit is treated as special.
- Two identical characters next to each other are a repeated-character violation (the first one found is reported).
- The prohibited words are 'password', 'qwerty' and 'admin' (case-insensitive).

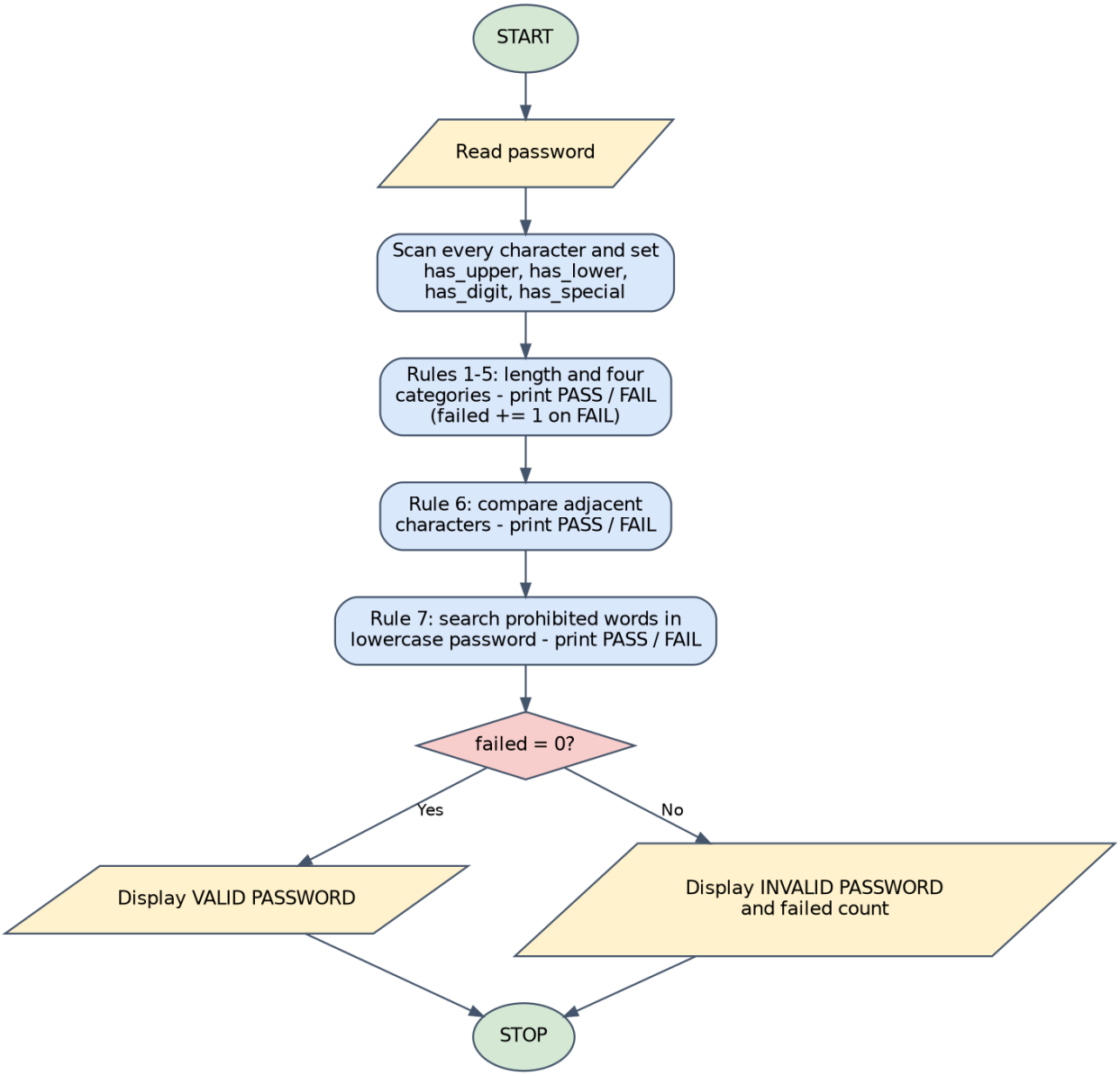
4. Algorithm

1. Read the password and set the four category flags to False.
2. Loop through every character and update the matching flag using string methods.
3. Check the length and print PASS or FAIL for each category rule.
4. Compare every character with the next one to find adjacent repeated characters.
5. Convert the password to lowercase and check each prohibited word using the in operator.
6. Count the failed rules and display VALID or INVALID with the number of failed rules.

5. Pseudocode

```
START
  SET min_length = 8
  SET prohibited_words = ["password", "qwerty", "admin"]
  READ password
  has_upper = has_lower = has_digit = has_special = FALSE
  FOR each character in password
    IF character is uppercase THEN has_upper = TRUE
    ELSE IF character is lowercase THEN has_lower = TRUE
    ELSE IF character is a digit THEN has_digit = TRUE
    ELSE has_special = TRUE
  END FOR
  failed = 0
  CHECK length, has_upper, has_lower, has_digit, has_special
    PRINT PASS, or PRINT FAIL and failed = failed + 1
  FOR i = 0 to length - 2
    IF password[i] = password[i + 1] THEN report repeated character, failed + 1
  END FOR
  FOR each word in prohibited_words
    IF word is in lowercase password THEN report word, failed + 1
  END FOR
  IF failed = 0 THEN PRINT "VALID PASSWORD"
  ELSE PRINT "INVALID PASSWORD", failed
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 29: Password Policy Validation Engine
min_length = 8
prohibited_words = ["password", "qwerty", "admin"]

password = input("Enter password: ")

has_upper = False
has_lower = False
has_digit = False
has_special = False

for character in password:
    if character.isupper():
        has_upper = True
    elif character.islower():
        has_lower = True
    elif character.isdigit():
        has_digit = True
    else:
        has_special = True

failed = 0
print("Rule check:")

if len(password) >= min_length:
```

```
    print("  [PASS] Minimum length of 8 characters")
else:
    print("  [FAIL] Password has only", len(password), "characters; minimum is 8")
    failed += 1

if has_upper:
    print("  [PASS] Contains an uppercase letter")
else:
    print("  [FAIL] Add at least one uppercase letter")
    failed += 1

if has_lower:
    print("  [PASS] Contains a lowercase letter")
else:
    print("  [FAIL] Add at least one lowercase letter")
    failed += 1

if has_digit:
    print("  [PASS] Contains a digit")
else:
    print("  [FAIL] Add at least one digit")
    failed += 1

if has_special:
    print("  [PASS] Contains a special character")
else:
    print("  [FAIL] Add at least one special character")
    failed += 1

# Adjacent repeated characters
repeat_found = False
for i in range(len(password) - 1):
    if password[i] == password[i + 1] and not repeat_found:
        repeat_found = True
        print(f"  [FAIL] Repeated character '{password[i]}' at positions {i + 1} and {i + 2}")
        failed += 1
if not repeat_found:
    print("  [PASS] No adjacent repeated characters")

# Prohibited words (case-insensitive)
lower_password = password.lower()
word_found = False
for word in prohibited_words:
    if word in lower_password:
        word_found = True
        print(f"  [FAIL] Prohibited word found: '{word}'")
        failed += 1
if not word_found:
    print("  [PASS] No prohibited words")

if failed == 0:
    print("Result: VALID PASSWORD")
else:
    print(f"Result: INVALID PASSWORD ({failed} rule(s) failed)")
```

8. Test Cases

Test Case	Password	Expected Result
1	Abc@1234	All rules pass - VALID PASSWORD
2	abc123	3 rules fail: length, uppercase letter, special character
3	Password@1	2 rules fail: repeated character 's' and prohibited word 'password'
4	Admin@2024	1 rule fails: prohibited word 'admin'
5	Hello@1122	1 rule fails: repeated character 'l' (positions 3 and 4)
6	ABCDEFG@1	1 rule fails: no lowercase letter

9. Sample Output

Sample Run 1 (input: Abc@1234)

```
Enter password: Abc@1234
Rule check:
  [PASS] Minimum length of 8 characters
  [PASS] Contains an uppercase letter
  [PASS] Contains a lowercase letter
  [PASS] Contains a digit
  [PASS] Contains a special character
  [PASS] No adjacent repeated characters
  [PASS] No prohibited words
Result: VALID PASSWORD
```

Sample Run 2 (input: Password@1)

```
Enter password: Password@1
Rule check:
  [PASS] Minimum length of 8 characters
  [PASS] Contains an uppercase letter
  [PASS] Contains a lowercase letter
  [PASS] Contains a digit
  [PASS] Contains a special character
  [FAIL] Repeated character 's' at positions 3 and 4
  [FAIL] Prohibited word found: 'password'
Result: INVALID PASSWORD (2 rule(s) failed)
```

Sample Run 3 (input: abc123)

```
Enter password: abc123
Rule check:
  [FAIL] Password has only 6 characters; minimum is 8
  [FAIL] Add at least one uppercase letter
  [PASS] Contains a lowercase letter
  [PASS] Contains a digit
  [FAIL] Add at least one special character
  [PASS] No adjacent repeated characters
  [PASS] No prohibited words
Result: INVALID PASSWORD (3 rule(s) failed)
```

10. Conclusion

The Password Policy Validation Engine uses strings, string methods, indexing, loops, lists and the membership operator to check every password rule independently. Reporting each failed rule with a clear message makes the program useful for explaining why a password is weak.

CASE STUDY 35 – TEXT COMPRESSION USING CHARACTER RUNS

1. Problem Statement

Design a Python program that compresses a text by replacing every group of consecutive identical characters with the character followed by the run length (for example, aaabb becomes a3b2). The program compares the original and encoded lengths and keeps the encoded text only when the compression is beneficial.

2. Objectives

1. Read a text and validate it.
2. Group consecutive repeated characters into runs.
3. Build the encoded string using string concatenation.
4. Compare the original and encoded lengths.
5. Handle cases where encoding is not beneficial.

3. Assumptions

- The text must not be empty and must not contain digits, so that the encoded string cannot be confused with run counts.
- Letter case is significant: 'a' and 'A' are different characters.
- Compression is beneficial only when the encoded text is shorter than the original text.
- If the encoded text is equal in length or longer, the original text is kept.

4. Algorithm

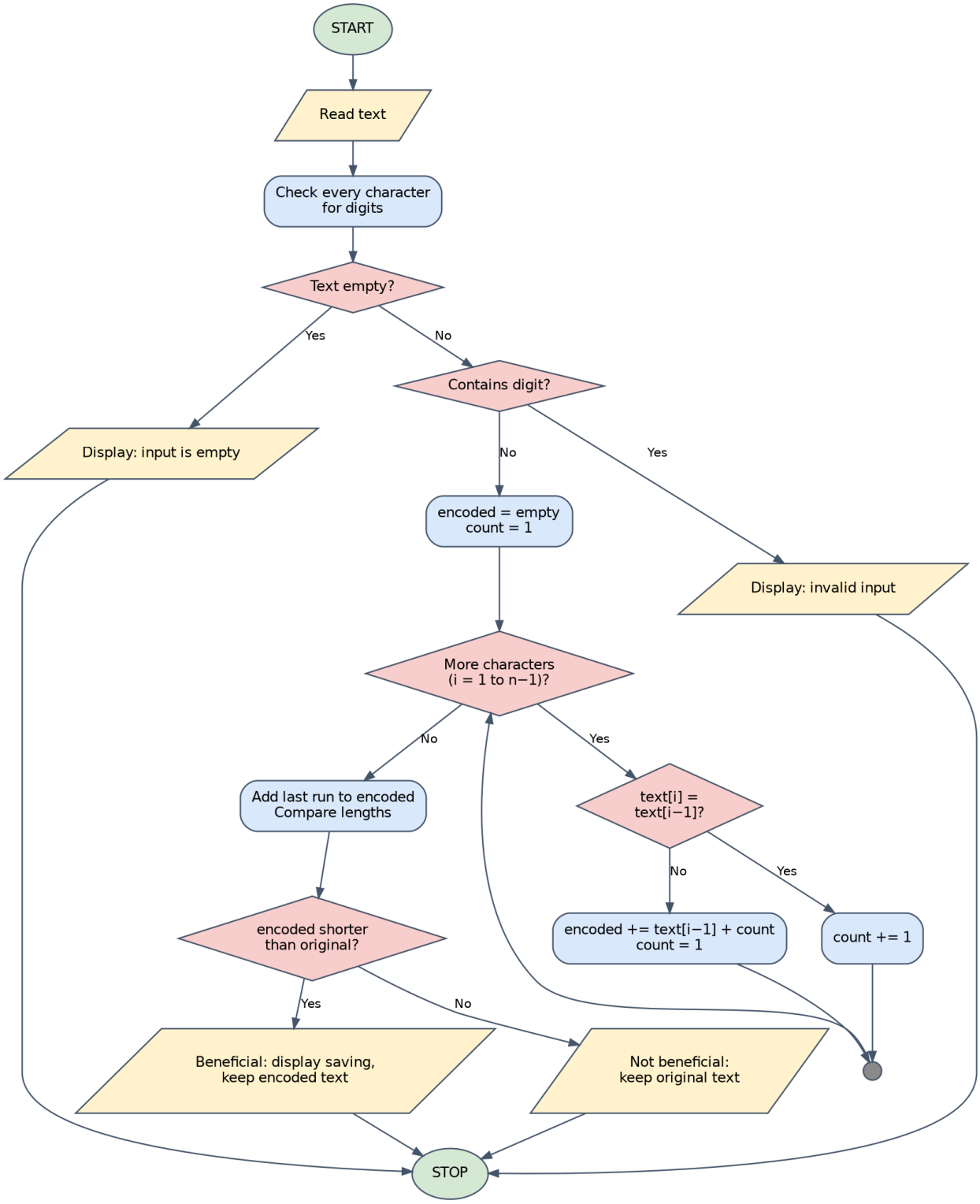
1. Read the text.
2. Reject an empty text or a text that contains a digit.
3. Start with an empty encoded string and count = 1.
4. Compare every character with the previous character: if they are equal, increase count; otherwise add the previous character and count to the encoded string and reset count to 1.
5. After the loop, add the last character and its count.
6. Calculate the original and encoded lengths.
7. If the encoded text is shorter, display the saving and keep it; otherwise keep the original text.

5. Pseudocode

```
START
  READ text
  has_digit = FALSE
  FOR each character in text
    IF character is a digit THEN has_digit = TRUE
  END FOR
  IF text is empty THEN
    PRINT "Input is empty"
  ELSE IF has_digit THEN
    PRINT "Invalid input"
  ELSE
    encoded = ""
    count = 1
    FOR i = 1 to length(text) - 1
      IF text[i] = text[i - 1] THEN
        count = count + 1
      ELSE
        encoded = encoded + text[i - 1] + count
        count = 1
      END IF
    END FOR
    encoded = encoded + last character + count
    IF length(encoded) < length(text) THEN
      PRINT saving and keep encoded text
    ELSE
      PRINT "Not beneficial" and keep original text
```

```
    END IF
  END IF
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 35: Text Compression Using Character Runs
text = input("Enter text to compress: ")

has_digit = False
for character in text:
    if character.isdigit():
        has_digit = True

if len(text) == 0:
    print("Input is empty. Nothing to compress.")
elif has_digit:
    print("Invalid input: text must not contain digits.")
else:
    encoded = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            encoded = encoded + text[i - 1] + str(count)
            count = 1

    # Add the last run
    encoded = encoded + text[len(text) - 1] + str(count)

    original_length = len(text)
    encoded_length = len(encoded)

    print("Original text  :", text)
    print("Encoded text   :", encoded)
    print("Original length:", original_length)
    print("Encoded length :", encoded_length)

    if encoded_length < original_length:
        saved = original_length - encoded_length
        percent = saved * 100 / original_length
        print(f"Result: Compression is beneficial ({saved} characters saved, {percent:.1f}%)")
        print("Final stored text:", encoded)
    else:
        print("Result: Compression is NOT beneficial; the original text is kept")
        print("Final stored text:", text)
```

8. Test Cases

Test Case	Input Text	Expected Result
1	aaabbbcccc	a3b3c4 (10 → 6 characters, 40.0% saved) - beneficial
2	aaaaaaaaaaaa (12 a's)	a12 (12 → 3 characters, 75.0% saved) - beneficial
3	abc	a1b1c1 (3 → 6 characters) - not beneficial, original kept
4	aabb	a2b2 (4 → 4 characters) - not beneficial, original kept
5	a	a1 (1 → 2 characters) - not beneficial, original kept
6	(empty text)	Input is empty. Nothing to compress.
7	ab12	Invalid input: text must not contain digits

9. Sample Output

Sample Run 1 (input: aaabbbcccc)

```
Enter text to compress: aaabbbcccc
Original text  : aaabbbcccc
Encoded text   : a3b3c4
Original length: 10
Encoded length : 6
Result: Compression is beneficial (4 characters saved, 40.0%)
Final stored text: a3b3c4
```

Sample Run 2 (input: abc)

```
Enter text to compress: abc
```

```
Original text  : abc
Encoded text   : a1b1c1
Original length: 3
Encoded length : 6
Result: Compression is NOT beneficial; the original text is kept
Final stored text: abc
```

Sample Run 3 (input: ab12)

```
Enter text to compress: ab12
Invalid input: text must not contain digits.
```

10. Conclusion

The Text Compression program uses strings, indexing, string concatenation and loops to implement run-length encoding. By comparing the original and encoded lengths, it also shows that compression is not always beneficial, which is an important idea in computational thinking.

CASE STUDY 42 – STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Design a Python program that analyses a matrix of student marks across subjects. The program calculates the total and average of every student (row-wise), the total, average and highest mark of every subject (column-wise), identifies the top performer and detects subjects with a low average.

2. Objectives

1. Represent the marks of all students as a nested list.
2. Calculate row-wise totals and averages.
3. Identify the top performer.
4. Calculate column-wise totals, averages and highest marks.
5. Detect subjects whose average is below the low-average limit.
6. Display the analysis in a formatted table.

3. Assumptions

- Each row of the matrix represents a student and each column represents a subject (Python, Maths, Web, English).
- All marks are between 0 and 100.
- A subject average below 60 is considered a low average.
- If two students have the same average, the student who appears first is the top performer.

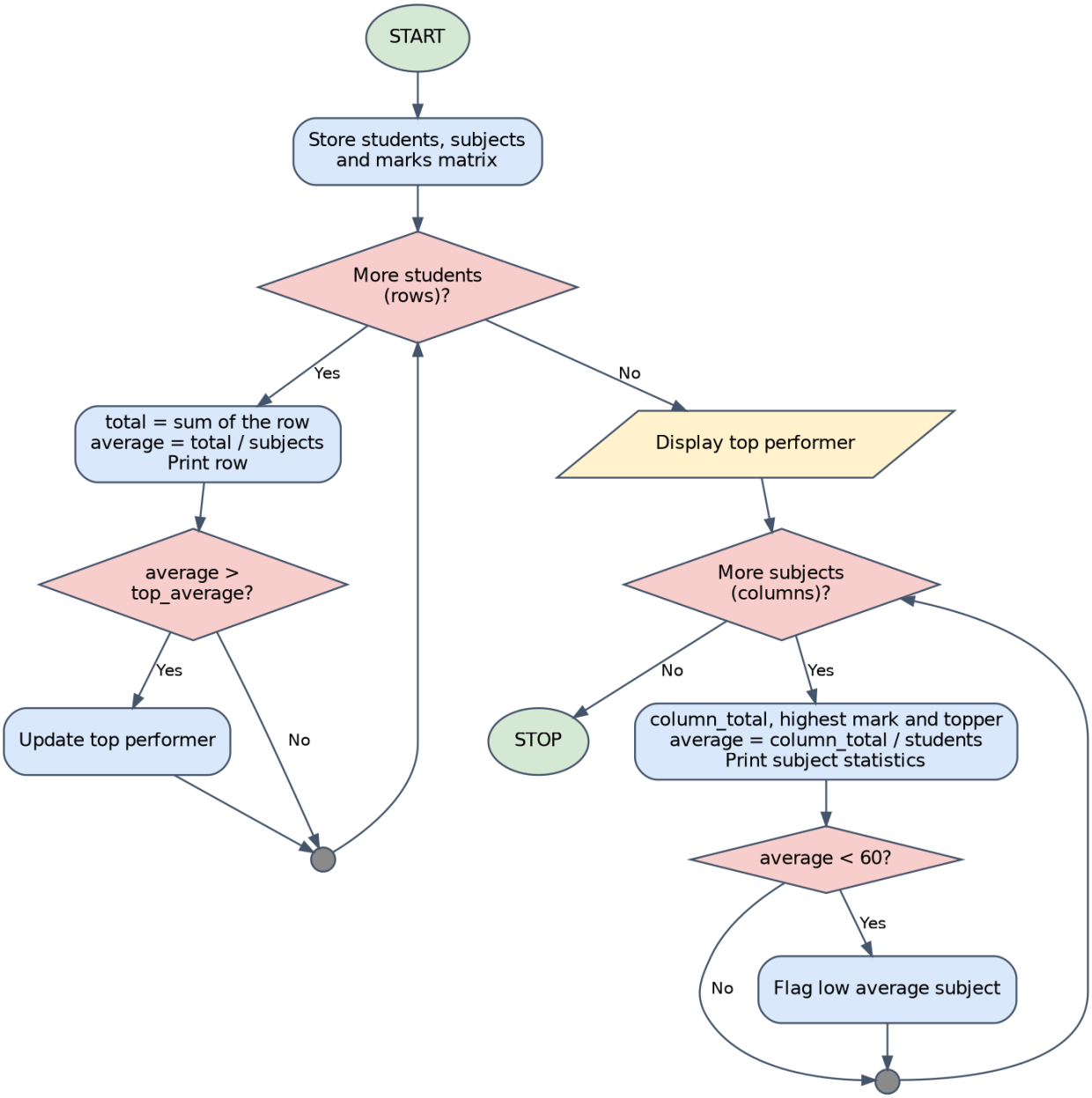
4. Algorithm

1. Store the student names, subject names and the marks matrix.
2. For every student (row), add the marks of all subjects to get the total and calculate the average.
3. Print the row and update the top performer when a higher average is found.
4. For every subject (column), add the marks of all students and find the highest mark and the topper.
5. Calculate the subject average and display the column statistics.
6. If the subject average is below 60, flag it as a low-average subject.

5. Pseudocode

```
START
  STORE students, subjects and marks (nested list)
  low_average = 60
  top_average = -1
  FOR each student row i
    total = 0
    FOR each subject column j
      total = total + marks[i][j]
    END FOR
    average = total / number of subjects
    PRINT row, total, average
    IF average > top_average THEN
      top_average = average
      top_student = students[i]
    END IF
  END FOR
  PRINT top_student
  FOR each subject column j
    column_total = 0
    highest = marks[0][j]
    FOR each student row i
      column_total = column_total + marks[i][j]
      IF marks[i][j] > highest THEN update highest and topper
    END FOR
    average = column_total / number of students
    PRINT column_total, average, highest, topper
    IF average < low_average THEN PRINT "Low average subject"
  END FOR
STOP
```

6. Flowchart



7. Python Source Code

```
# Case Study 42: Student Marks Matrix Analysis
students = ["Anu", "Bala", "Chandu", "Divya", "Esha"]
subjects = ["Python", "Maths", "Web", "English"]
low_average = 60

marks = [
    [80, 75, 90, 70],
    [65, 45, 70, 60],
    [90, 88, 85, 92],
    [55, 40, 48, 62],
    [72, 38, 65, 58]
]

# ---------- Row-wise analysis (students) ----------
print("STUDENT-WISE RESULT")
print("-" * 54)
print(f"{'Student':<10}{'Python':>8}{'Maths':>7}{'Web':>6}{'English':>9}{'Total':>7}{'Avg':>7}")

top_student = ""
top_average = -1

for i in range(len(students)):
    total = 0
```

```
    for j in range(len(subjects)):
        total += marks[i][j]
    average = total / len(subjects)

    print(f"{students[i]:<10}{marks[i][0]:>8}{marks[i][1]:>7}{marks[i][2]:>6}{marks[i][3]:>9}{total:>7}{average:>7.2f}"

    if average > top_average:
        top_average = average
        top_student = students[i]

print("-" * 54)
print(f"Top performer: {top_student} (average {top_average:.2f})")

# ---------- Column-wise analysis (subjects) ----------
print()
print("SUBJECT-WISE ANALYSIS")
print("-" * 54)

for j in range(len(subjects)):
    column_total = 0
    highest = marks[0][j]
    topper = students[0]

    for i in range(len(students)):
        column_total += marks[i][j]
        if marks[i][j] > highest:
            highest = marks[i][j]
            topper = students[i]

    average = column_total / len(students)
    print(f"{subjects[j]:<9} Total: {column_total:<4} Average: {average:<6.2f} Highest: {highest} ({topper})")

    if average < low_average:
        print(f"          >> Low average subject (below {low_average})")
```

8. Test Cases

Test Case	Row / Column	Expected Result
1	Student Anu (80, 75, 90, 70)	Total 315, average 78.75
2	Student Divya (55, 40, 48, 62)	Total 205, average 51.25
3	Top performer	Chandu (average 88.75)
4	Subject Maths	Total 286, average 57.20, highest 88 (Chandu) - flagged as low average
5	Subject Python	Total 362, average 72.40, highest 90 (Chandu)
6	Subject English	Total 342, average 68.40, highest 92 (Chandu) - not low

9. Sample Output

```
STUDENT-WISE RESULT
------------------------------------------------------
Student     Python  Maths   Web  English  Total    Avg
Anu             80     75    90       70    315  78.75
Bala            65     45    70       60    240  60.00
Chandu          90     88    85       92    355  88.75
Divya           55     40    48       62    205  51.25
Esha            72     38    65       58    233  58.25
------------------------------------------------------
Top performer: Chandu (average 88.75)

SUBJECT-WISE ANALYSIS
------------------------------------------------------
Python    Total: 362  Average: 72.40  Highest: 90 (Chandu)
Maths     Total: 286  Average: 57.20  Highest: 88 (Chandu)
          >> Low average subject (below 60)
Web       Total: 358  Average: 71.60  Highest: 90 (Anu)
English   Total: 342  Average: 68.40  Highest: 92 (Chandu)
```

10. Conclusion

The Student Marks Matrix program uses nested lists and nested loops to perform row-wise and column-wise analysis. It shows how indexing, comparison operators and formatted output can be combined to find totals, averages, toppers and low-performing subjects.

CASE STUDY 49 – TUPLE-BASED PRODUCT CATALOGUE AUDIT

1. Problem Statement

Design a Python program that audits a product catalogue stored as tuples of product ID, price and stock quantity. The program detects invalid and duplicate records, calculates the inventory value of valid products and identifies the products that need replenishment.

2. Objectives

1. Store catalogue records as tuples in a list.
2. Detect invalid prices, invalid stock quantities and duplicate product IDs.
3. Calculate the inventory value of every valid product and the total value.
4. Identify low-stock products and calculate the units to order.
5. Display an audit report with a summary.

3. Assumptions

• Each tuple contains (product ID, price, stock).
• Price must be greater than zero and stock cannot be negative.
• A product ID that has already appeared in a valid record is a duplicate and is rejected.
• Stock below 5 units is low stock; replenishment orders raise the stock to a target of 20 units.

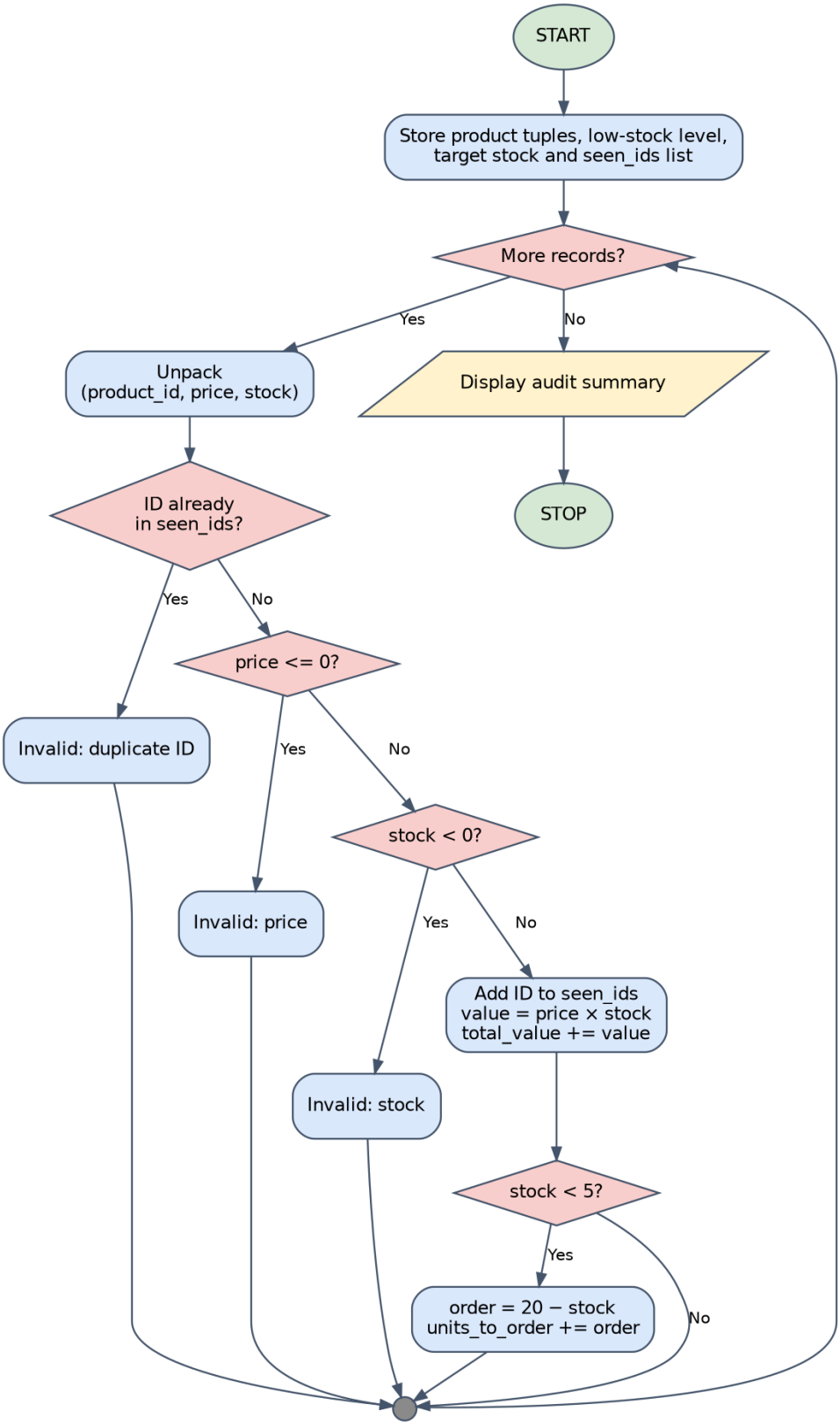
4. Algorithm

1. Store the product tuples in a list and create an empty list of seen product IDs.
2. Unpack each tuple into product ID, price and stock.
3. If the ID is already in the seen list, report a duplicate; else if the price is not positive or the stock is negative, report an invalid record.
4. For a valid record add the ID to the seen list, calculate value = price × stock and add it to the total.
5. If stock < 5, calculate order = 20 − stock and add it to the units to replenish.
6. Display the valid count, invalid count, total stock value and units to replenish.

5. Pseudocode

```
START
  SET low_stock_level = 5, target_stock = 20
  STORE products as (product_id, price, stock)
  seen_ids = empty list
  total_value = 0, valid_count = 0, invalid_count = 0, units_to_order = 0
  FOR each (product_id, price, stock) in products
    IF product_id is in seen_ids THEN
      PRINT "duplicate product ID"
      invalid_count = invalid_count + 1
    ELSE IF price <= 0 THEN
      PRINT "price must be greater than zero"
      invalid_count = invalid_count + 1
    ELSE IF stock < 0 THEN
      PRINT "stock cannot be negative"
      invalid_count = invalid_count + 1
    ELSE
      add product_id to seen_ids
      valid_count = valid_count + 1
      value = price * stock
      total_value = total_value + value
      IF stock < low_stock_level THEN
        order = target_stock - stock
        units_to_order = units_to_order + order
        PRINT replenishment message
      END IF
    END IF
  END FOR
  PRINT valid_count, invalid_count, total_value, units_to_order
STOP
```

6. Flowchart


START
Store product tuples, low-stock level, target stock and seen_ids list
More records?
Yes
No
Unpack (product_id, price, stock)
Display audit summary
STOP
ID already in seen_ids?
Yes
No
price <= 0?
Invalid: duplicate ID
Yes
No
stock < 0?
Invalid: price
Yes
No
Add ID to seen_ids value = price × stock total_value += value
Invalid: stock
stock < 5?
Yes
No
order = 20 − stock units_to_order += order

7. Python Source Code

```
# Case Study 49: Tuple-Based Product Catalogue Audit
low_stock_level = 5
target_stock = 20

# (Product ID, Price, Stock)
products = [
    ("P101", 100, 12),
    ("P102", 250, 3),
    ("P103", 80, 0),
    ("P104", -20, 5),
    ("P105", 150, 8),
    ("P101", 100, 12),
    ("P106", 60, -4)
]

seen_ids = []
total_value = 0
valid_count = 0
invalid_count = 0
units_to_order = 0

print("PRODUCT CATALOGUE AUDIT")
print("-" * 58)

for product_id, price, stock in products:
    if product_id in seen_ids:
        print(f"{product_id}: INVALID - duplicate product ID")
        invalid_count += 1
    elif price <= 0:
        print(f"{product_id}: INVALID - price must be greater than zero")
        invalid_count += 1
    elif stock < 0:
        print(f"{product_id}: INVALID - stock cannot be negative")
        invalid_count += 1
    else:
        seen_ids.append(product_id)
        valid_count += 1
        value = price * stock
        total_value += value
        print(f"{product_id}: Price {price}, Stock {stock}, Value {value}")

        if stock < low_stock_level:
            order = target_stock - stock
            units_to_order += order
            print(f"      >> Low stock: order {order} units to reach {target_stock}")

print("-" * 58)
print("Valid records      :", valid_count)
print("Invalid records    :", invalid_count)
print("Total stock value  :", total_value)
print("Units to replenish :", units_to_order)
```

8. Test Cases

Test Case	Record (ID, price, stock)	Expected Result
1	P101, 100, 12	Valid; value 1200
2	P102, 250, 3	Valid; value 750; low stock - order 17 units
3	P103, 80, 0	Valid; value 0; low stock - order 20 units
4	P104, -20, 5	Invalid: price must be greater than zero
5	P105, 150, 8	Valid; value 1200
6	P101, 100, 12 (again)	Invalid: duplicate product ID
7	P106, 60, -4	Invalid: stock cannot be negative

9. Sample Output

```
PRODUCT CATALOGUE AUDIT
------------------------------------------------------------
P101: Price 100, Stock 12, Value 1200
P102: Price 250, Stock 3, Value 750
      >> Low stock: order 17 units to reach 20
P103: Price 80, Stock 0, Value 0
      >> Low stock: order 20 units to reach 20
P104: INVALID - price must be greater than zero
P105: Price 150, Stock 8, Value 1200
P101: INVALID - duplicate product ID
P106: INVALID - stock cannot be negative
------------------------------------------------------------
Valid records      : 4
Invalid records    : 3
Total stock value  : 3150
Units to replenish : 37
```

10. Conclusion

The Product Catalogue Audit program stores records as tuples and uses tuple unpacking, the membership operator and conditional statements to separate valid and invalid records. It calculates inventory values and replenishment quantities, showing how lists and tuples can model real catalogue data.